CONTENTS

Advanced AI systems vocabulary IBM Technology

Seven Terms That Describe a Modern AI System

Place agents, reasoning models, vector search, RAG, MCP, mixture of experts and ASI on one map, and see how they connect.

For engineers who keep meeting these terms and want the mechanism behind each, not the marketing · 26 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- What a large language model does at the input/output level
- A rough idea of what a neural network layer is
- Comfort reading Python

Overview

These seven terms are not a list of unrelated buzzwords. Six of them describe layers of one system, and they stack: a mixture-of-experts architecture makes a large model affordable to serve, reasoning fine-tuning turns that model into one that plans, an agent loop turns the planner into something that acts, MCP gives it a standard way to reach external systems, and retrieval over a vector database gives it facts it was never trained on.

The seventh, artificial superintelligence, is different in kind. It names a hypothesis rather than a component, and it is worth keeping separate from the six precisely because it is not something you can build with today.

What follows takes each term down to the mechanism — what it actually does, what it costs, and where it breaks — because most of these words are used loosely enough that knowing the definition and knowing the thing are quite different states.

KEY TAKEAWAYS

- ✓ An agent is a loop — perceive, reason, act, observe — around a model that is otherwise stateless and reactive.
- ✓ Large reasoning models are fine-tuned to work step by step, trained on problems whose answers can be checked automatically.
- ✓ A vector database stores embeddings so that 'find similar' becomes a distance computation rather than a keyword match.
- ✓ RAG uses that search to inject relevant text into the prompt, giving a model facts it was never trained on.
- ✓ MCP standardises how applications expose tools and data to models, replacing one-off connectors with a common protocol.
- ✓ Mixture of experts activates only a few subnetworks per token, so total parameters and active parameters come apart.
- ✓ MoE saves computation, not memory — every expert still has to be resident even though few are used.
- ✓ AGI and ASI are both theoretical; ASI additionally supposes recursive self-improvement, which is what makes it a different claim.

01 Agentic AI

0:40

> Systems that reason and act autonomously toward a goal, cycling through perceive, reason, act and observe instead of answering once.

An AI agent reasons and acts autonomously to achieve a goal. Unlike a chatbot, which responds to one prompt at a time, an agent runs on its own through a cycle: it perceives its environment, reasons about the best next step, acts on the plan it has built, observes the result, and goes round again.

The roles this enables are broad because the loop is generic. A travel agent that books a trip. A data analyst spotting trends across quarterly reports. A DevOps engineer detecting anomalies in logs, spinning up containers to test fixes, and rolling back faulty deployments.

What is worth holding onto is that none of this is a property of the model. The model is stateless and reactive; it answers when asked. Everything agentic is the program around it — the loop, the tools it can call, the message history that carries results forward. Swap in a different model and you still have an agent; remove the loop and you have a chatbot, whatever model you kept.

That framing tells you where the engineering effort goes, and it is not where people expect. A loop has no natural stopping point, so termination is your problem: a step budget, a check for repeated actions, and a defined outcome when the budget runs out. Actions with consequences need limits enforced in code rather than requested in the prompt, because a probabilistic system will occasionally do the improbable thing.

KEY POINTS

- → The cycle is perceive → reason → act → observe, repeated until the goal is met.
- → A chatbot answers once; an agent pursues an objective across many steps.
- → Nothing agentic lives in the model — the loop, the tools and the history are ordinary code.
- → The loop has no inherent stopping condition, so budgets and termination are your responsibility.
- → Consequential actions need limits enforced in code, not merely requested in the prompt.

The loop, in the smallest honest form

Three of the four stages are plain software. Only 'reason' is the model. The step budget is not defensive decoration — nothing in the model guarantees it will ever decide it is finished.

```
def run(goal, tools, max_steps=20):
    history = [system_prompt(), user(goal)]

    for _ in range(max_steps):
        reply = model.chat(history, tools=tools)   # reason
        if not reply.tool_calls:
            return reply.content                   # it says it is done

        history.append(reply)
        for call in reply.tool_calls:              # act
            try:
                result = tools[call.name](**call.arguments)
            except Exception as e:
                result = f"ERROR: {e}"             # failures are observations
            history.append(tool_result(call.id, result))   # observe

    raise BudgetExceeded(goal)
```

The same model, two different systems

Worth stating plainly because the marketing rarely does: whether something is an agent is a question about your code, not about which model you bought.

```
chatbot   prompt ──> model ──> answer
                               (done)

agent     goal ──> [ model ──> tool ──> result ] ──> answer
                        ^                   |
                        └───────────────────┘
                            until done
```

02 Large reasoning models

1:23

> LLMs given reasoning-focused fine-tuning so they work through a problem step by step before answering.

Agents are typically built on a particular kind of large language model: a large reasoning model. These are LLMs that have undergone reasoning-focused fine-tuning. Where a regular model generates its response immediately, a reasoning model is trained to work through the problem step by step — which is exactly what an agent needs when planning a complex, multi-step task.

The training is the interesting part, and it explains both the strength and the limits. The model is trained on problems with verifiably correct answers: mathematics, and code that a compiler or test suite can check. Through reinforcement learning it learns to generate reasoning sequences that lead to correct final answers. The reward is not human preference about whether the reasoning sounds good — it is whether the answer was right.

That requirement for automatic verification is the constraint worth carrying. It is why these models improved fastest on maths and code, and why domains without a programmatic checker benefit less directly. If you cannot write the grader, you cannot run this training loop.

When a chatbot pauses and says it is thinking, that is the reasoning model generating an internal chain of thought, breaking the problem down before producing a response.

The practical cost is that thinking is generated text, and generated text is tokens, latency and money — paid before the user sees a single word of the answer. Reasoning depth is a budget decision per call, not a setting to turn on and forget. For a hard planning step it pays for itself; for classifying a support ticket it is waste.

KEY POINTS

- → Reasoning models are LLMs fine-tuned to work step by step before answering.
- → Training uses problems with verifiably correct answers — maths, and code a compiler can test.
- → Reinforcement learning rewards reaching the correct answer, not producing plausible-sounding reasoning.
- → Domains without an automatic checker gain less, because the training loop needs a grader.
- → The visible 'thinking' pause is chain-of-thought being generated.
- → Reasoning tokens cost latency and money on every call, so depth is a per-task budget decision.

Why verifiable answers make the training loop work

The grader is the whole design. Where one can be written, the model optimises the property you actually care about. Where it cannot, you are back to rewarding text that reads well — which is a different and much weaker signal.

```
def reward(problem, attempt):
    """Programmatic and definitive — this is what makes the loop possible."""
    if problem.kind == "math":
        return float(parse_final(attempt) == problem.answer)
    if problem.kind == "code":
        if not compiles(attempt):
            return 0.0
        return passed(attempt) / problem.total_tests

# works: maths, code, anything with a checkable answer
# does not: "write a good summary" — no definitive right answer
```

Thinking is not free

Illustrative figures, not measurements, but the shape is real: reasoning tokens are generated before the answer begins, so they are latency the user waits through and tokens you are billed for.

```
standard model    prompt ──> 180 output tokens          ~1.2 s
reasoning model   prompt ──> 2,400 thinking tokens
                           + 180 output tokens          ~9.0 s

# worth it: planning a multi-step migration
# not worth it: routing a support ticket into one of six categories
```

03 Vector databases

2:46

> Data stored as embeddings rather than raw blobs, so similarity search becomes a mathematical operation.

In a vector database you do not store raw data — text files, images — as opaque blobs. An embedding model converts the data into a vector: a long list of numbers that captures the semantic meaning of the content.

The payoff is that search becomes arithmetic. Looking for embeddings that sit close to one another translates directly into finding semantically similar content. Take a picture of a mountain vista, run it through the embedding model to get a multidimensional numeric vector, and a similarity search returns the nearest vectors in that space. The same machinery works for text articles and audio.

What makes this different from a keyword index is that nothing is being matched. Two documents about the same subject that share no vocabulary still land near each other, because the embedding model was trained so that context determines position.

Two practical points follow immediately. Vectors are only comparable within the model that produced them, so embedding your corpus with one model and your queries with another yields valid-looking distances and meaningless results. And comparing a query against millions of vectors exhaustively is too slow, so real systems use approximate nearest-neighbour indexes that trade a small, measurable amount of recall for a very large amount of speed — measurable being the operative word, since that recall is a ceiling on everything built above it.

KEY POINTS

- → An embedding model converts content into a vector encoding its meaning.
- → Similarity search is a distance computation, not a keyword match.
- → Documents sharing no words can still rank as similar.
- → The same approach covers text, images and audio.
- → Vectors are only comparable within the model that produced them.
- → Approximate nearest-neighbour indexes trade a little recall for a lot of speed.

Similarity as geometry

The top match shares no content word with the query. That is the behaviour a keyword index cannot produce, and it is the entire reason vector search exists.

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

query = embed("revenue growth last quarter")

for doc in corpus:
    print(f"{cosine(query, embed(doc)):.3f}  {doc}")

# 0.91  "Q4 performance exceeded plan across all regions."
# 0.88  "Quarterly sales rose against the prior period."
# 0.22  "The revenue recognition policy is in appendix B."
#         ^ shares the word "revenue" and is still least relevant
```

The mixed-model bug

Nothing errors. Dimensions match, distances compute, results come back ranked — and they are noise. Store the model identity next to the index and assert on it, because this failure is otherwise very hard to see.

```
store.upsert(id=doc.id, vector=model_a.embed(doc.text))   # months ago
hits = store.search(model_b.embed(question), k=5)         # today

assert store.meta["embedding_model"] == model.name
```

04 Retrieval augmented generation

4:09

> Use vector search to pull relevant passages and paste them into the prompt, so the model answers from facts it was never trained on.

RAG uses a vector database to enrich the prompt sent to an LLM. The flow starts at a retriever component. It takes the user's prompt, turns it into a vector with an embedding model, and performs a similarity search against the vector database. What comes back is inserted into the original prompt, and the enriched prompt goes to the model.

So you can ask a question about company policy, and the system pulls the relevant section of the employee handbook into the prompt before the model ever sees the question.

The important structural fact is that the model is untouched. Same weights, same endpoint. RAG is a prompt-construction technique with a search engine in front of it — which is exactly why adding a document to the corpus makes it available on the very next query, with no retraining and no deployment.

The characteristic failure follows from the same structure. If retrieval returns the wrong passages, the model answers confidently from the wrong passages, and nothing raises an error. This has a name — silent failure — and it is invisible to ordinary monitoring, since the response is fluent and the status code is 200. The only way to see it is to build a set of questions whose correct source passage you already know, and measure how often retrieval actually returns it.

KEY POINTS

- → The retriever embeds the query, searches, and injects the best matches into the prompt.
- → The model's weights are never modified — RAG is entirely input-side.
- → New knowledge is live as soon as a document is indexed.
- → The model can only answer from passages it was given.
- → Silent failure: retrieval misses, the model answers from the wrong text, nothing errors.
- → Detect it with a gold set of questions whose correct source you already know.

The whole pipeline

Four steps, and the instruction to admit ignorance is what converts a confident guess into an honest refusal when retrieval comes back empty-handed.

```
def answer(question, store, model, k=5):
    hits = store.search(embed(question), k=k)          # 1. retrieve

    context = "\n\n".join(f"[{i+1}] {h.text}"          # 2. augment
                          for i, h in enumerate(hits))
    prompt = (
        "Answer using only the context below. If it does not "
        "contain the answer, say you do not know.\n\n"
        f"<context>\n{context}\n</context>\n\n{question}"
    )
    return model.generate(prompt)                      # 3. generate
```

Measuring the ceiling

Whatever this number tops out at is the best your whole system can do — no prompt engineering recovers a passage the model was never given. It is worth knowing before tuning anything downstream.

```
GOLD = [
    ("what is the parental leave policy?", "handbook.md#leave"),
    ("how do I expense a taxi?",           "finance.md#travel"),
]

def recall_at_k(store, k):
    hits = sum(any(h.source == want for h in store.search(embed(q), k=k))
               for q, want in GOLD)
    return hits / len(GOLD)

for k in (1, 3, 5, 10, 25):
    print(f"recall@{k:<3} {recall_at_k(store, k):.0%}")
```

05 Model Context Protocol

5:33

> A standard way for applications to expose tools and data to models, replacing bespoke connectors with one protocol.

For language models to be genuinely useful they have to reach external data sources, services and tools. MCP — Model Context Protocol — standardises how applications provide that context.

The problem it solves is combinatorial. Without a standard, connecting models to systems means building a one-off integration for each pairing: this model to that database, this assistant to that code repository, this tool to that email server. Every new tool multiplies the work. MCP replaces that with a common interface — write an MCP server once for a system, and any MCP-speaking client can use it.

An MCP server is what sits in front of the external system and tells the model exactly what is available and how to reach it. The client, typically the application hosting the model, connects to it.

The piece worth understanding beyond the diagram is what a server actually exposes, because it is not only functions. Tools are actions the model can invoke. Resources are data the client can read — files, records, documents — addressed by URI. Prompts are reusable templates the server offers. Keeping those distinct matters in practice: something the model should read is a resource, and something it should do is a tool, and conflating them produces servers that make the model call a function to fetch data it could simply have been given.

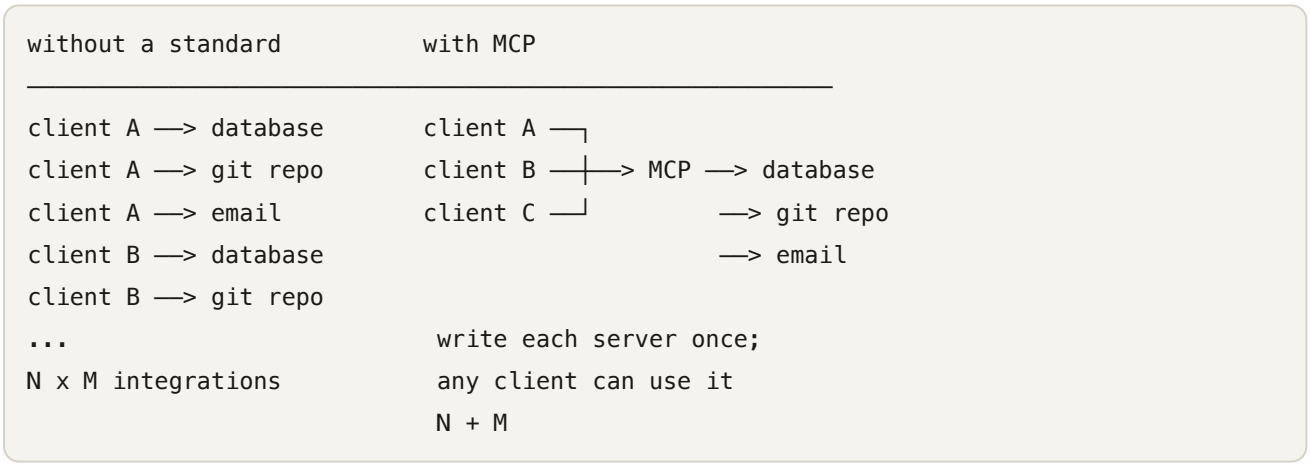
Security deserves a sentence, since MCP is precisely the boundary where a model reaches your systems. A server defines what is reachable at all, so it is the right place to enforce scope and permission — not the prompt, which is a request rather than a guarantee.

KEY POINTS

- → MCP standardises how applications expose context, tools and data to models.
- → It replaces one-off connectors per model-tool pairing with a single protocol.
- → An MCP server fronts an external system; the model's host application is the client.
- → Servers expose tools (actions), resources (readable data) and prompts (templates).
- → Something to read is a resource; something to do is a tool — conflating them is a common design error.
- → The server boundary is where scope and permission should be enforced.

Why a protocol rather than connectors

The arithmetic is the argument. Bespoke integration work grows with the product of clients and systems; a protocol makes it grow with the sum.



The three things a server exposes

Sketched in Python for readability rather than as any particular SDK's syntax. The distinction that matters is the annotation, not the function body: one is a read, the other is an action with consequences.

```
# a TOOL — an action the model may invoke
@tool("create_ticket",
      description="Open a support ticket. Use when the user reports a fault.")
def create_ticket(title: str, severity: Literal["low", "high"]) -> str:
    return tickets.create(title=title, severity=severity).id

# a RESOURCE — data the client can read, addressed by URI
@resource("tickets://open")
def open_tickets() -> str:
    return format_table(tickets.list(status="open"))

# a PROMPT — a reusable template the server offers
@prompt("triage")
def triage(ticket_id: str) -> str:
    return f"Triage ticket {ticket_id}. Assign severity and an owner."
```

Enforce scope at the server

The prompt can ask the model to behave; only the server can guarantee it. Anything with real consequences belongs behind a check the model cannot talk its way past.

```
@tool("delete_ticket")
def delete_ticket(ticket_id: str, *, caller) -> str:
    if not caller.has_scope("tickets:write"):
        return "REFUSED: this connection is read-only"
    if tickets.get(ticket_id).status == "closed":
        return "REFUSED: closed tickets are retained for audit"
    return tickets.delete(ticket_id)
```

06 Mixture of experts

6:15

> Split the model into specialised subnetworks and activate only the few each token needs, so size and cost come apart.

Mixture of experts is an old idea — the paper dates to 1991 — and it has never been more relevant than now.

MoE divides a large language model into a series of experts: specialised neural subnetworks. There might be three, or well over a hundred. A routing mechanism activates only the experts needed for a particular task, and a merge step combines their outputs mathematically into a single representation that continues through the rest of the model.

The result is an efficient way to scale model size without a proportional increase in compute. MoE models such as IBM Granite's 4.0 series can have dozens of experts, but for any given token only the specific experts needed are activated. So a model may have billions of total parameters while using only a fraction as active parameters at inference time.

Two details are worth adding because they are where MoE surprises people.

The first is that routing happens per token, not per task. Each token is scored by a small gating network, the top few experts are selected, and their outputs are combined weighted by those scores. Different tokens in the same sentence go to different experts.

The second is the one that changes deployment decisions: **MoE saves computation, not memory**. Every expert must be resident and ready, because you cannot know in advance which tokens will route where. A model with 100 billion total and 10 billion active parameters needs memory for 100 billion and compute for 10 billion. It is cheaper to run, not cheaper to host — which is exactly backwards from what 'only uses a fraction of its parameters' suggests on first hearing.

KEY POINTS

- → MoE splits the model into specialised expert subnetworks.
- → A router activates only the experts a given token needs, then merges their outputs.
- → Model size scales without a proportional increase in compute.
- → Routing is per token, not per task — one sentence uses many experts.
- → Total parameters and active parameters are different numbers.
- → Memory is sized by total parameters; compute by active ones. MoE saves FLOPs, not VRAM.

Routing and merging, per token

The gate is a small learned layer producing one score per expert. Only the top few run, and their outputs are summed weighted by those scores — which is what makes the merge differentiable and therefore trainable end to end.

```
def moe_layer(x, experts, gate, k=2):
    scores = softmax(gate(x))              # one score per expert
    top = topk_indices(scores, k)          # only these actually run

    out = 0
    for i in top:
        out = out + scores[i] * experts[i](x)   # weighted merge
    return out

# 128 experts defined, 2 evaluated per token.
# the other 126 still occupy memory.
```

Total versus active parameters

Illustrative numbers to show the shape. The middle column decides what hardware you need; the right column decides what a token costs. Reading only the right column is how teams end up with a model they cannot fit.

```
                   total params   active per token   what it means
──────────────────────────────────────────────────────────────────────
dense model             30 B             30 B          all of it, always
MoE model              100 B             10 B          3x the compute
                                                        efficiency ...

memory required        100 B worth                     ... but you host
                                                        all 100 B
```

Why routers need a balancing pressure

Left alone, a router tends to collapse onto a few favourites — they get more gradient, improve faster, and get chosen still more often. An auxiliary loss penalising uneven load is what keeps the remaining experts in use.

```
# without balancing: the router's rich get richer
expert usage: [41%, 33%, 18%, 5%, 2%, 1%, 0%, 0%]
               ^^^^^^^^^^ doing nearly all the work

# an auxiliary loss penalises uneven load
aux = alpha * n_experts * sum(frac_tokens[i] * mean_gate[i]
                              for i in range(n_experts))
loss = task_loss + aux
```

07 AGI and ASI

8:17

> Two theoretical thresholds. AGI matches human experts across cognitive tasks; ASI goes beyond, with recursive self-improvement.

Artificial superintelligence is the stated goal of the frontier labs, and at this point it is purely theoretical. It does not exist, and we do not know whether it ever will.

Today's best models are slowly approaching a different standard: artificial general intelligence. AGI is also theoretical, but the definition is specific — a system that could complete all cognitive tasks as well as any human expert. Note both quantifiers, because they are doing the work. All tasks, not most. As well as an expert, not as well as an average person.

ASI is a step beyond: intellectual scope past human-level intelligence, potentially capable of recursive self-improvement. That last clause is what makes ASI a different kind of claim rather than simply a larger number. A system that can redesign and upgrade itself becomes smarter, and the smarter version is better at redesigning itself, in a cycle with no obvious ceiling.

It is the kind of development that would either solve humanity's biggest problems or create entirely new ones nobody has imagined.

For an engineer, the practical value of these two terms is mostly in noticing when they are being used to describe something that has been measured. Benchmark results are narrow and specific: this model scored this on that set of problems. AGI is a claim about generality across all cognitive work, which no benchmark tests. When you see the words attached to a product, the useful question is which specific capability was measured, on what, and by whom — because the gap between a strong benchmark score and 'all cognitive tasks as well as any expert' is not a gap of degree.

KEY POINTS

- → ASI is theoretical, does not exist, and may never.
- → AGI is also theoretical: all cognitive tasks, as well as any human expert.
- → Both quantifiers matter — all tasks, and expert-level rather than average.
- → ASI additionally supposes recursive self-improvement, which is a different kind of claim.
- → Benchmarks measure narrow capabilities; generality is not among them.
- → When the terms appear in marketing, ask what was measured, on what, by whom.

The three levels, and what separates them

Only the first row describes anything that exists. The distinction between the second and third is not scale but self-modification, which is why they are separate terms rather than points on one axis.

```
narrow AI      strong at specific tasks; everything shipping today
                 ↓
AGI            all cognitive tasks, at the level of a human expert
                 ↓            (theoretical)
ASI            beyond human level, plus recursive self-improvement
                              (theoretical)

# the last step is not "more" — it is a system that can rewrite itself
```

What recursive self-improvement means

The reason ASI is discussed as a discontinuity rather than a milestone. Written as a loop, the claim is easy to state and the missing piece is obvious: nothing today can perform the improve step on itself.

```
system = current_best()

while True:
    better = system.redesign(system)   # the step nobody can do today
    if not better.more_capable_than(system):
        break
    system = better

# each iteration produces something better at running the next one.
# no known system closes this loop; the whole idea rests on it.
```

Glossary

Agentic AI

A system that pursues a goal autonomously by cycling through perceiving, reasoning, acting and observing, rather than answering a single prompt.

Large reasoning model

An LLM fine-tuned to work through problems step by step before answering, trained on tasks whose answers can be checked automatically.

Chain of thought

The intermediate reasoning a model generates before its final answer. What a chatbot is producing while it shows that it is thinking.

Embedding model

A model that converts text, images or audio into a vector whose position encodes the content's meaning.

Vector database

Storage built to hold embeddings and retrieve them by similarity, turning 'find things like this' into a distance computation.

RAG

Retrieval augmented generation: searching a corpus for relevant passages and inserting them into the prompt so the model can answer from them.

Silent failure

When retrieval misses the passage containing the answer. No error is raised — the model simply answers from the wrong material, fluently.

MCP

Model Context Protocol: a standard for applications to expose tools, data and prompt templates to models, replacing per-pairing connectors.

MCP server

The component fronting an external system, declaring what a model can reach there and how. The right place to enforce permission.

Tool vs resource

In MCP, a tool is an action the model invokes; a resource is data the client reads. Something to do versus something to read.

Mixture of experts (MoE)

An architecture splitting a model into specialised subnetworks, with a router activating only a few per token.

Router / gating network

The small learned layer that scores experts for each token and selects the top few to run.

Active parameters

The fraction of a MoE model's parameters used for a given token. Determines compute cost, while total parameters determine memory.

AGI

Artificial general intelligence: a theoretical system able to complete all cognitive tasks as well as any human expert.

ASI

Artificial superintelligence: theoretical capability beyond human level, potentially able to redesign and improve itself recursively.

Check yourself

Answer first, then open each one.

Two products use the same model. One is a chatbot and one is an agent. What actually differs?

The control flow around the call. The agent puts the model in a loop with tools and feeds each result back into context so the next turn can use it. Nothing about the model changed, which is why 'is this agentic?' is a question about your code rather than about which model you bought.

Why did reasoning models improve fastest on mathematics and code specifically?

Because their reinforcement learning needs a reward signal, and those domains have verifiable answers — a result that is right or wrong, code that compiles and passes tests. Where you cannot write a programmatic grader you cannot run the loop, so progress there is less direct.

A RAG system returns a fluent, confident, wrong answer, and the logs show no errors. What has probably happened, and why won't monitoring catch it?

Silent failure: retrieval returned plausible but incorrect passages, and the model answered from them. There is nothing to alert on because the request succeeded and the response is well formed. Detecting it requires a gold set of questions with known source passages and a recall measurement you run deliberately.

Why does MCP change integration effort from a product into a sum?

Without a standard, each client-system pairing needs its own connector, so work grows as clients times systems. With MCP each system needs one server and each client speaks the protocol once, so work grows as clients plus systems — and a new client gets every existing server for free.

An MoE model has 100B total and 10B active parameters. What does each number tell you, and which one is easy to misread?

Active parameters determine compute per token, so the model runs roughly as cheaply as a 10B dense model. Total parameters determine memory, because every expert must be resident — you cannot predict which tokens route where. The easy misreading is 'only uses a fraction of its parameters', which sounds like a hosting saving and is not: MoE saves FLOPs, not VRAM.

Why is ASI treated as a different kind of claim from AGI rather than simply more of it?

Because of recursive self-improvement. AGI names a capability level — all cognitive tasks at expert standard. ASI supposes a system that can redesign and upgrade itself, so each version is better at producing the next. That is a qualitative difference in the shape of the claim, not a larger point on the same scale.

Where to go next

- → Write an MCP server for one internal system your team uses, and notice how much of the work is deciding what should be a tool versus a resource rather than writing the integration.
- → Read the original mixture-of-experts paper (Jacobs, Jordan, Nowlan and Hinton, 1991) and then a modern sparse-MoE transformer paper, and note that the routing idea barely changed while the scale did.
- → Instrument expert utilisation on an open MoE model and look for collapse — a few experts taking most of the tokens is the failure the balancing loss exists to prevent.
- → Take a task you currently send to a reasoning model and measure whether the thinking tokens change the answer; for classification and extraction they frequently do not, and the latency is pure cost.
- → Build the gold-set recall measurement for a RAG system you already run, before touching any prompt — that number is the ceiling on everything above it.
- → Trace one agent run end to end and mark which steps were the model and which were ordinary code; the ratio is usually a surprise and tells you where bugs will actually live.
- → The vector-database, RAG and agent entries here each have a full lesson elsewhere in this collection — use this page as the map and those for the depth.
